SEPM PROJECT

ONLINE DIGITAL SHOPPING FASHION

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Submitted by

Names : Trupti Avhadkar (26)
Suhani Parate (25)
Section : B4
Batch : B2

Document Control and Contents

Revision	Date	Description	Approval
1.0	25 Oct 2026	Initial proposed requirements baseline	Pending academic review

Table of Contents

Section	Content
1	Introduction
2	Overall Description
3	Specific Requirements
4	UML Behavioral and Structural Models
5	Architecture, Data and Interfaces
6	Verification and Validation
7	Feasibility, Risks and Roadmap
9	Detailed Operational Contracts
10	Conclusion and References

List of Figures

Figures 1–8: Use Case, Activity, Class, Sequence, Order State, Deployment, Component and Entity Relationship Diagrams.

List of Tables

Tables 1–15: document control, comparison, feature baseline, requirement catalogue, quality attributes, permissions, data dictionary, input/output rules, traceability, test cases, feasibility and risk register.

Reading convention: SHALL indicates a mandatory requirement; SHOULD indicates a recommended future enhancement. All performance targets are proposed acceptance criteria, subject to deployment sizing.

1. Introduction

1.1 Purpose of the Document

This SRS establishes a testable agreement between stakeholders, developers, designers and evaluators for the proposed Digital Fashion Shopping System (DFSS). It specifies functions and constraints without prescribing every implementation detail.

1.2 Scope of the Project

DFSS supports discovery and purchase of fashion products through a responsive customer storefront and protected administrator console. The baseline covers accounts, catalogues, variants, search, wishlist, cart, coupons, checkout, third-party payment, order lifecycle, inventory, reviews, shipment tracking, notifications and reporting. A virtual try-on prototype is proposed as an optional future integration, not a committed production capability.

1.3 Project Objectives

Provide consistent cross-device shopping; prevent overselling; minimize checkout friction; ensure auditable order and payment states; support controlled returns and cancellations; protect customer information; expose operational insights to administrators.

1.4 Intended Audience

Customers, administrators, academic reviewers, product owners, UI/UX designers, backend engineers, database engineers, QA testers, security reviewers and integration partners.

1.5 Definitions, Acronyms and Abbreviations

Term	Definition
SKU	Stock keeping unit; unique purchasable size/colour variant.
PII	Personally identifiable information.
RBAC	Role-based access control.
API	Application programming interface.
RTM	Requirements traceability matrix.
OTP	One-time password where supported.
PCI DSS	Payment-card security standard applicable to payment providers.
RPO / RTO	Recovery point objective / recovery time objective.

1.6 References

IEEE Std 830-1998 (historical SRS guidance); ISO/IEC/IEEE 29148:2018 (requirements engineering); OWASP ASVS; WCAG 2.2; applicable payment-provider integration documentation. See Section 8 for bibliographic details.

1.7 Document Overview

Chapter 2 describes context and actors; Chapter 3 provides numbered verifiable requirements; Chapters 4–5 model behavior, architecture and data; Chapter 6 establishes validation; Chapter 7 assesses feasibility and risk.

2. Overall Description

2.1 Product Perspective

DFSS is a proposed three-tier web platform: presentation clients invoke a stateless application API, which persists transactional records in a relational database and communicates with external payment, shipping and notification providers.

2.2 Existing System and 2.3 Limitations

Conventional fragmented retail journeys may rely on disconnected catalogues, manual stock updates, inconsistent variant selection and delayed delivery status. These are comparison assumptions rather than claims about a named competitor.

Dimension	Typical fragmented workflow	Proposed DFSS
Inventory	Manual reconciliation	Atomic SKU-level reservation
Discovery	Basic keyword navigation	Faceted category, size, colour and price filters
Checkout	Disjoint payment/order records	Verified gateway callbacks and idempotent order updates
Operations	Manual summaries	Role-restricted dashboard and sales reports

2.4 Proposed System

A responsive storefront is backed by centralized catalogue, pricing, stock and order services. An administrator manages publishable listings, variants, fulfilment and refund decisions. Integrations are isolated behind adapter interfaces.

2.5 Product Features and Release Baseline

Capability	Baseline / proposed extension
Registration and login	Baseline SHALL
Product catalogue and filters	Baseline SHALL
Size/colour variants	Baseline SHALL
Wishlist and cart	Baseline SHALL
Coupons and checkout	Baseline SHALL
Payment verification	Baseline SHALL
Tracking and returns	Baseline SHALL
Reviews and notifications	Baseline SHALL
Admin inventory and analytics	Baseline SHALL
Virtual try-on	Future / optional SHOULD
AI personalized stylist	Future / optional SHOULD
Multilingual interface	Future / optional SHOULD

2.6 User Classes and Characteristics

Guest: views catalogue; Customer: owns account and places orders; Administrator: manages products, stock, orders and reports; External services: payment, delivery and notification integrations with narrowly scoped API access.

2.7 Operating Environment

Modern desktop/mobile browsers with TLS 1.2+; responsive web UI; application deployed on Linux-based cloud or equivalent; PostgreSQL-compatible relational database; object storage for product media; managed HTTPS reverse proxy.

2.8 Design and Implementation Constraints

All monetary amounts use integer minor units and currency code; SKU stock cannot be negative; payment card details are entered only on provider-hosted/tokenized interfaces; sensitive endpoints require authentication and authorization; transactional writes require database constraints and audit events.

2.9 Assumptions and Dependencies

Customers have internet access; administrators maintain accurate product metadata; gateways and carriers publish supported APIs; payment confirmation is asynchronous; tax and consumer return rules are configured for the deployment jurisdiction; shipment updates may be delayed by external providers.

2.10 User Characteristics

Customer interactions SHALL accommodate keyboard navigation and mobile touch; administrator workflows assume basic catalogue and fulfilment training. User-facing errors SHALL use plain language without leaking implementation details.

3. Specific Requirements

3.1 Functional Requirements

Each requirement is uniquely identified and independently testable. “Input → processing → output” is specified below; priorities are Must (M) or Should (S). Dependencies reference prerequisite requirement IDs or external services.

ID / name	Input	Processing / description	Expected output	Priority / dependency
FR-01 Account registration	Email, password	Validate uniqueness and policy	Verified or pending account	M / Email service
FR-02 Authentication	Credentials	Verify hash; issue session	Authenticated session	M / FR-01
FR-03 Profile management	Address/profile fields	Authorize and validate update	Saved customer profile	M / FR-02
FR-04 Catalogue browse	Category/page	Load published products	Paginated product cards	M / Product DB
FR-05 Search and filters	Query/facets/sort	Apply combined criteria	Matching variants/products	M / FR-04

ID / name	Input	Processing / description	Expected output	Priority / dependency
FR-06 Variant selection	SKU size/colour	Resolve price and stock	Variant availability	M / FR-04
FR-07 Wishlist	Product ID	Deduplicate customer item	Updated wishlist	M / FR-02
FR-08 Cart management	SKU/quantity	Validate and recalculate	Updated cart totals	M / FR-06
FR-09 Coupon application	Coupon code/cart	Check eligibility and expiry	Discount or rejection	M / FR-08
FR-10 Checkout	Address/cart	Reprice and reserve stock	Pending order/payment intent	M / FR-08, FR-09

ID / name	Input	Processing / description	Expected output	Priority / dependency
FR-11 Payment verification	Signed gateway event	Verify signature/idempotency	Confirmed or failed payment	M / FR-10, gateway
FR-12 Order confirmation	Verified payment	Commit order; send receipt	Order ID and notification	M / FR-11
FR-13 Shipment tracking	Order/carrier event	Map carrier statuses	Visible tracking timeline	M / FR-12, carrier
FR-14 Cancellation/return	Order/reason	Check policy; initiate workflow	Status and refund request	M / FR-12
FR-15 Reviews and ratings	Delivered SKU/rating	Check verified purchase	Published moderated review	S / FR-13

ID / name	Input	Processing / description	Expected output	Priority / dependency

FR-16 Admin catalogue	Product and media	RBAC and validation	Published/updated listing	M / FR-02
FR-17 Inventory control	SKU/stock adjustment	Audit and concurrency check	Accurate available stock	M / FR-16
FR-18 Analytics	Date range	Aggregate orders/refunds	Restricted sales report	S / FR-12
FR-19 Notifications	Order events	Queue templated message	Delivery attempt/log	M / FR-12
FR-20 Recommendations	Browse/order context	Rank eligible products	Suggestion list	S / FR-04

3.2 Non-Functional Requirements

ID	Measurable acceptance criterion
NFR-01	95% of catalogue GET requests complete ≤ 2 s at 200 concurrent users under stated load.
NFR-02	99.5% monthly service availability, excluding approved maintenance.
NFR-03	No duplicate order confirmation for repeated identical gateway event IDs.
NFR-04	100% of privileged API endpoints enforce server-side RBAC.
NFR-05	TLS 1.2+ in transit; encryption at rest for PII and backups.
NFR-06	All critical transactional failures logged with correlation ID and no secrets.
NFR-07	Daily backups; target RPO ≤ 24 h and RTO ≤ 4 h in recovery rehearsal.
NFR-08	Support 10,000 active SKUs and 200 concurrent users in acceptance load test.
NFR-09	Core checkout accessible by keyboard; target WCAG 2.2 AA.
NFR-10	Automated unit tests cover ≥ 80% of business-rule branches.
NFR-11	Search returns deterministic pagination and stable sort for identical data.
NFR-12	P95 checkout API processing ≤ 3 s excluding third-party payment latency.

3.3 Hardware Requirements

Client: internet-connected smartphone or desktop with at least 2 GB available RAM. Illustrative server starting capacity: 2 vCPU/4 GB RAM for application, managed relational database with 4 GB RAM and SSD storage; scale after load measurements. These are planning assumptions, not mandatory vendor specifications.

3.4 Software Requirements

Responsive HTML/CSS/JavaScript client; REST/JSON API runtime; PostgreSQL-compatible RDBMS; reverse proxy; container runtime and CI pipeline; secure secrets manager; payment, carrier and messaging adapters.

3.5 External Interface Requirements

HTTPS JSON APIs SHALL use versioned routes, authenticated access and structured errors; payment webhooks SHALL validate signatures, event identifiers and timestamps; delivery integration SHALL map provider statuses to internal shipment states; notification adapters SHALL accept templated event payloads without exposing credentials.

3.6 User Interface Requirements

Navigation includes catalogue, search, product details, wishlist, cart, checkout, account, orders and admin console. Show stock and variant errors inline, explicit total/tax/shipping before payment, visible loading state, accessible labels and confirmation of irreversible actions.

3.7 Database Requirements

Relational integrity SHALL link order items to immutable purchase snapshots, payment attempts to orders and variants to products. Unique constraints SHALL protect email, SKU, coupon code and gateway event ID. Transactional reservation prevents negative available stock.

3.8 Security Requirements

Salted adaptive password hashing; secure HttpOnly SameSite session cookies or equivalent; rate limits for login; CSRF controls where cookie-authenticated; input parameterization; RBAC; audit logs for admin stock/price/refund changes; data minimization and retention policies.

3.9–3.12 Performance, Reliability, Scalability and Maintainability

NFR-01/08/12 define load acceptance. Retry-safe payment and notification processing protects consistency. Services SHOULD be horizontally scalable without sticky application state. Modules SHALL expose documented contracts, schema migrations, observability and automated regression tests.

4. UML Behavioral and Structural Models

The following figures model the proposed baseline. External actors are separate trust boundaries. Optional features are explicitly identified as extensions. Diagram semantics take precedence over decorative layout.

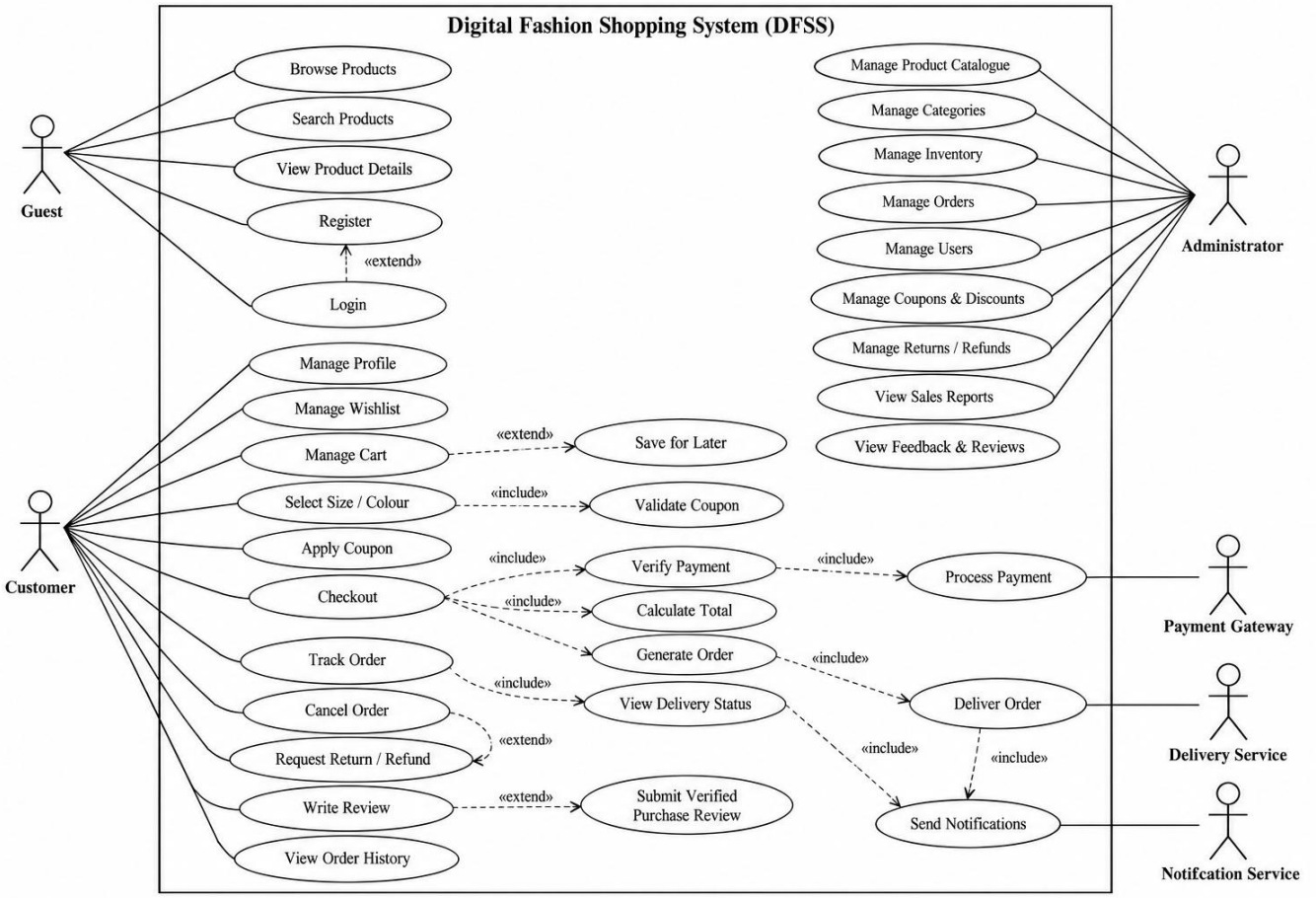


Figure 1. Use case model

Customer and administrator are primary actors; gateway, carrier and messaging systems are supporting actors. Checkout includes payment verification and coupon validation, while a return is conditional on an existing eligible order.

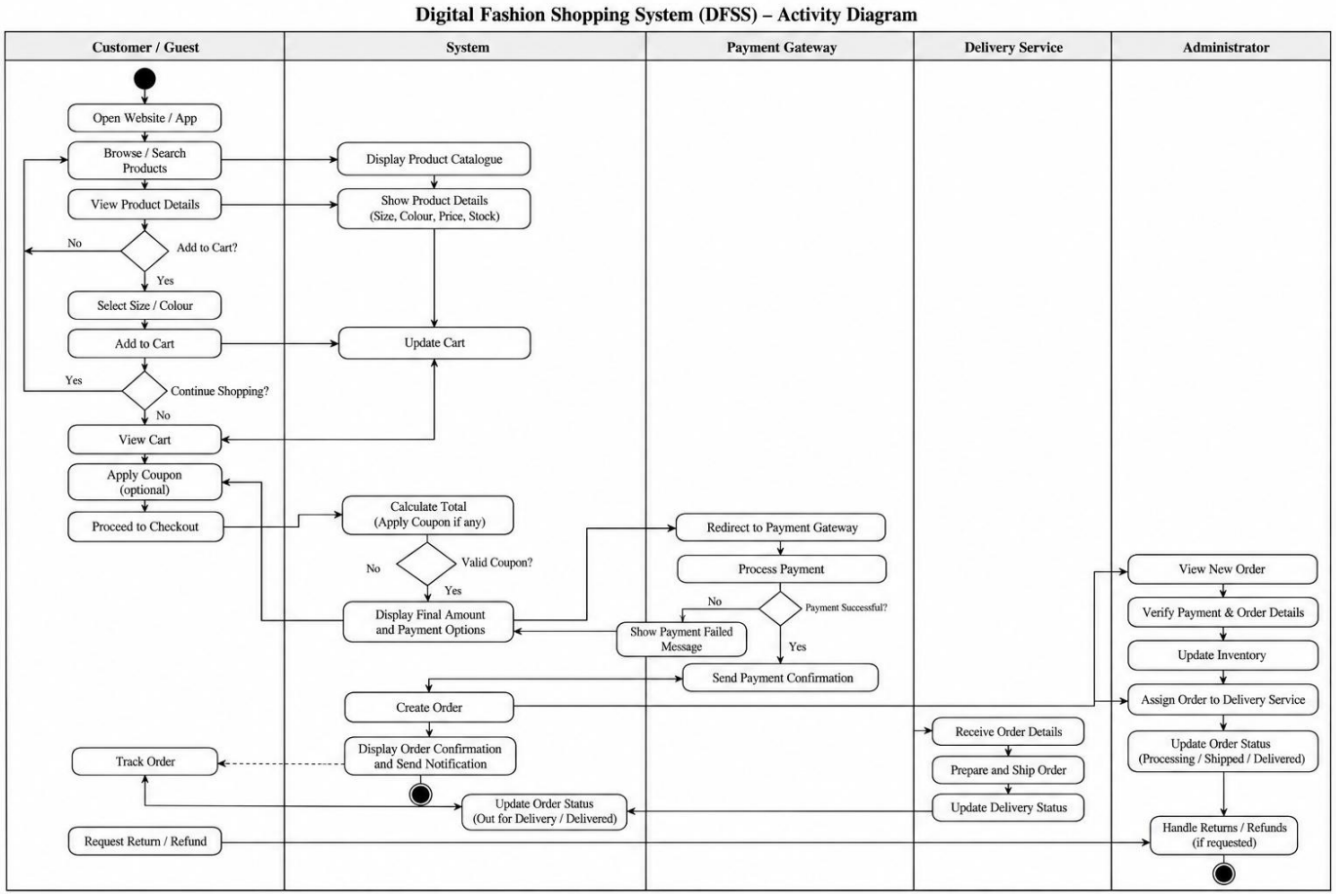


Figure 2. Customer purchase activity

A stock check precedes cart acceptance and a second authoritative validation occurs at checkout. Failure releases reservations; only verified payment advances an order to confirmed.

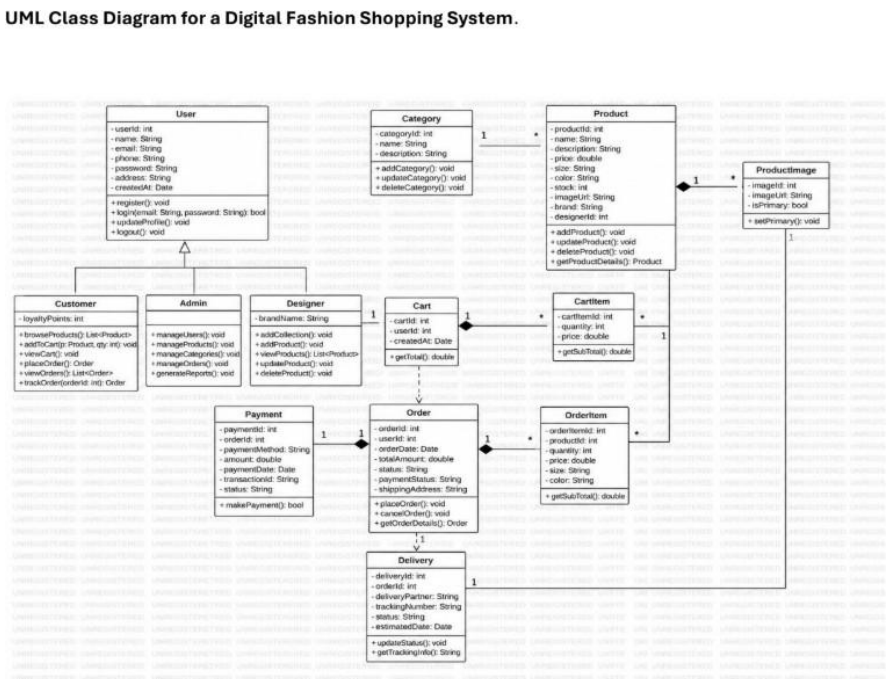


Figure 3. Domain class diagram

Inheritance separates customer and administrator behavior. Filled diamonds indicate ownership of cart and order lines. Product prices are snapshotted in OrderItem to preserve historical totals.

Sequence Diagram : Digital Fashion Shopping System

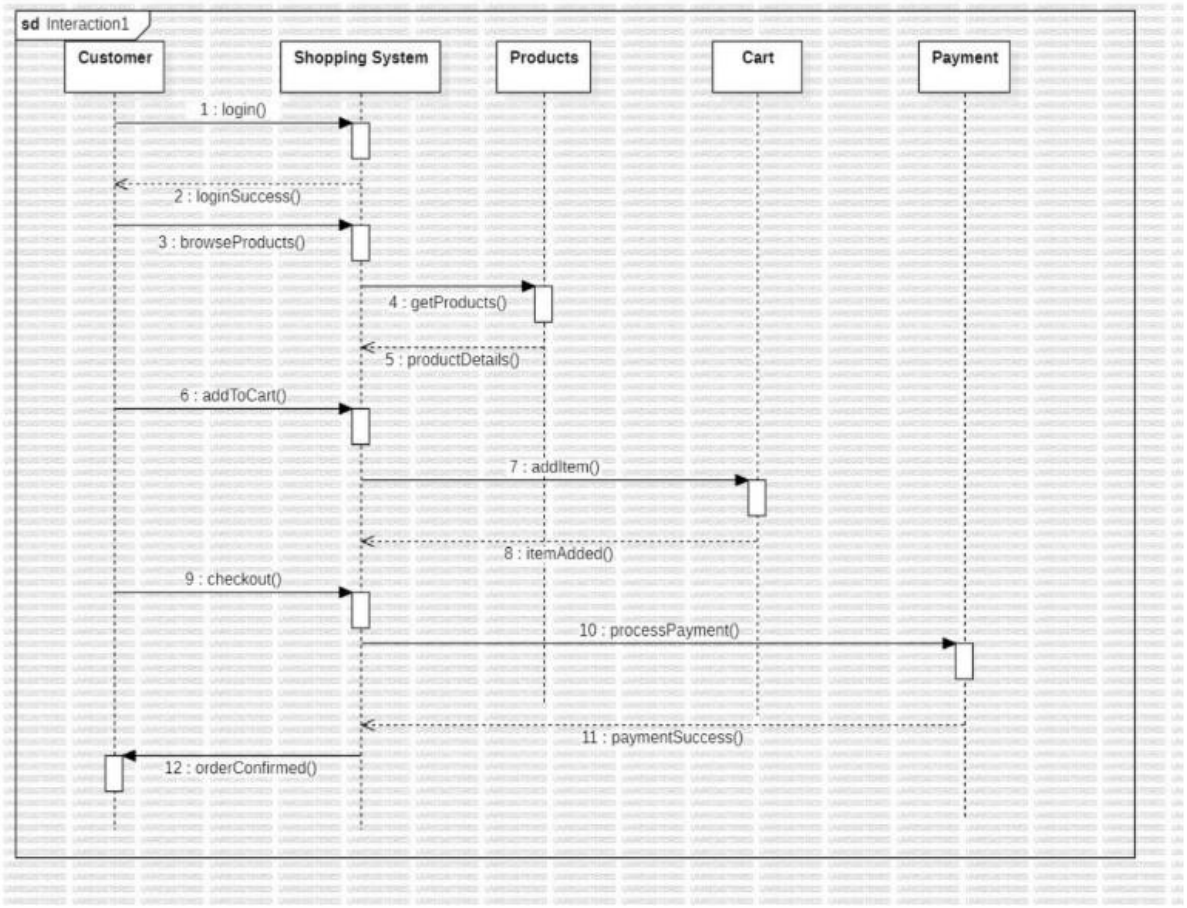


Figure 4. Order placement sequence

Numbered calls convey temporal order; dashed edges denote replies or asynchronous gateway/notification events. Alternative outcomes are success (confirm and notify) or failure (release stock and retain a failed attempt).

Digital Fashion Shopping System (DFSS) – State Diagram (Order Lifecycle)

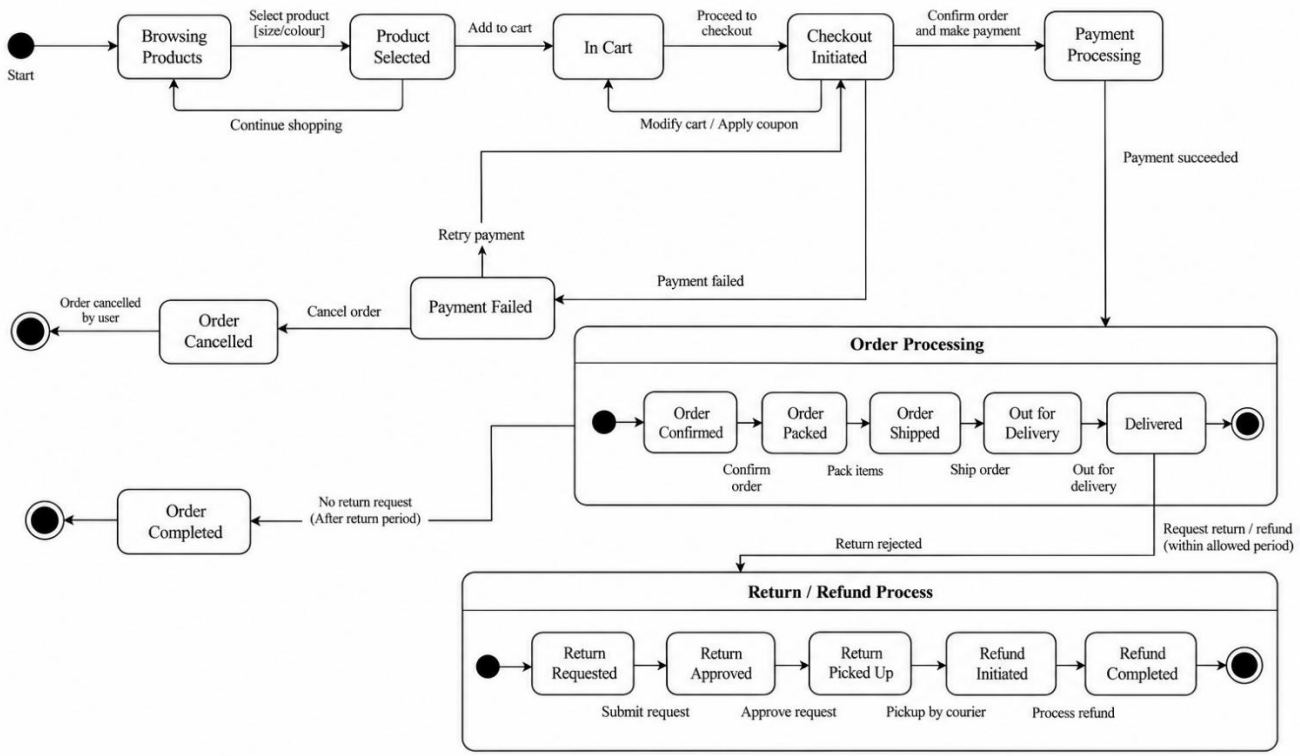


Figure 5. Order lifecycle state machine

Transitions are guarded by payment verification, fulfilment stage and configured return policy. A failed payment is not a confirmed order; refund applies only where funds were captured.

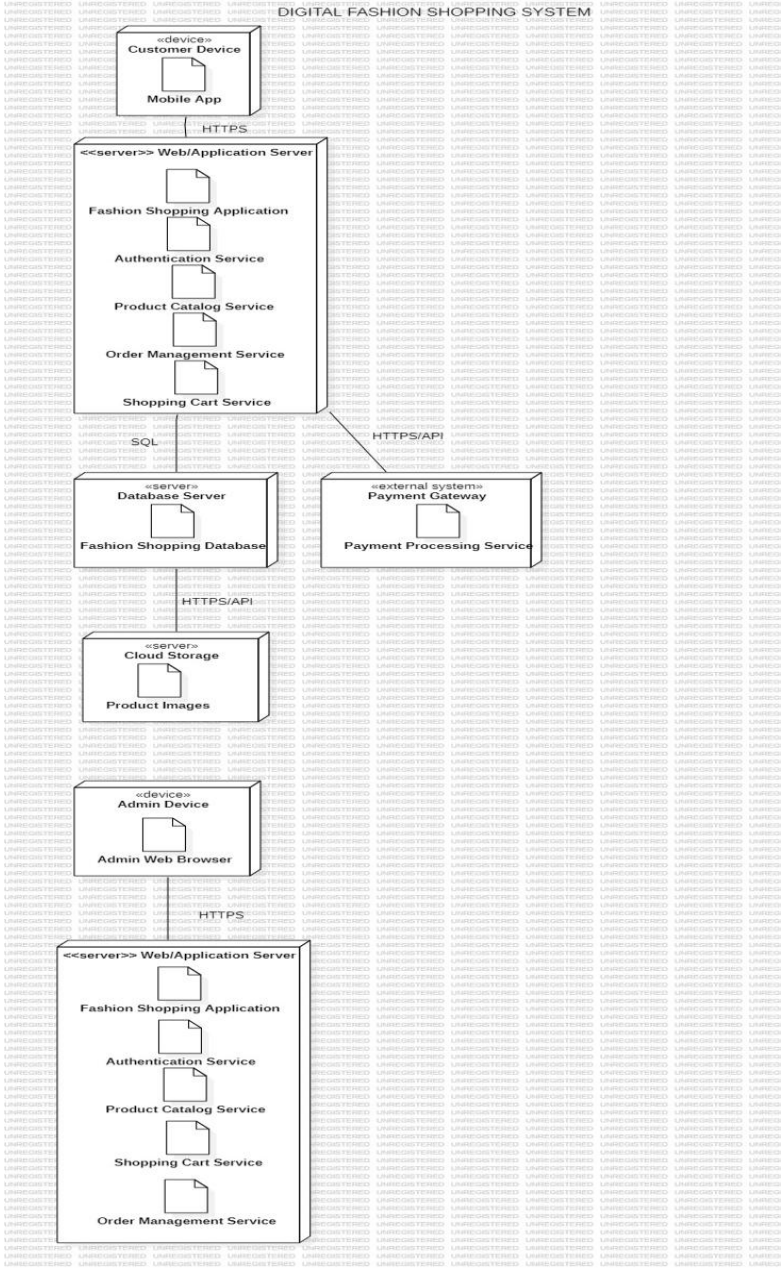


Figure 6. Deployment architecture

Trust boundaries separate customer and administrator devices from the application and external vendors. Secrets reside server-side; database access is private, and gateway webhooks are verified before state changes.

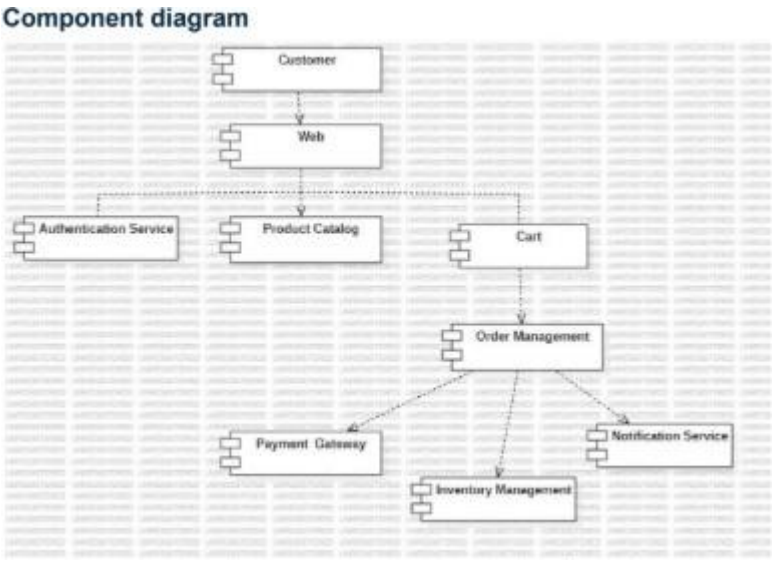


Figure 7. Logical component architecture

Required interfaces are labelled on dependency edges; provider implementations expose product query, stock reservation, payment and notification capabilities. The UI never accesses the database directly.

5. Architecture, Data and Interfaces

5.1 Three-tier architecture

Presentation tier: accessible responsive storefront and admin views. Application tier: authorization, product search, cart calculation, order state machine, payment webhook and background notifications. Data tier: relational records, transactional constraints, audit history and object-stored media.

5.2 Data dictionary

Entity / field	Type / constraint	Purpose
Customer.email	VARCHAR(254), UNIQUE, NOT NULL	Login identifier
Variant.sku	VARCHAR(64), UNIQUE	Size/colour sellable unit
Inventory.available	INTEGER CHECK ≥ 0	Unreserved sellable quantity
Inventory.reserved	INTEGER CHECK ≥ 0	Pending checkout allocation
Order.status	ENUM / constrained text	Lifecycle state
Order.total_minor	BIGINT CHECK ≥ 0	Final monetary total
Payment.provider_event_id	VARCHAR(128), UNIQUE	Webhook deduplication
OrderItem.unit_price_minor	BIGINT CHECK ≥ 0	Immutable purchase price
Review.rating	SMALLINT CHECK 1–5	Customer rating
Coupon.expires_at	TIMESTAMPTZ	Eligibility boundary

5.3 Roles and permissions

Operation	Guest	Customer	Administrator
Browse/search	Yes	Yes	Yes

Wishlist/cart/checkout	No	Own only	No
Read orders	No	Own only	All authorized
Review product	No	Eligible purchases	Moderate
Manage products/inventory	No	No	Yes
Sales reports/refund decision	No	No	Yes

5.4 Input and output specifications

Input	Validation	Output / error
Email/password	Email syntax, uniqueness, password policy	Account/session or field error
SKU and quantity	Existing active SKU; integer 1–10; stock	Cart subtotal or availability error
Coupon	Code, validity, minimum spend, usage limit	Applied discount or reason
Checkout address	Required fields, supported region	Pending order and payment intent
Payment webhook	Signature, event ID, amount/currency match	Idempotent status update
Review	Delivered purchase; rating 1–5; length cap	Moderation/published state

ID	Scenario	Preconditions	Test steps	Test data	Expected result	Actual result	Status

5.5 Validation and error handling

Return 400 for malformed input, 401 for missing authentication, 403 for insufficient permission, 404 for absent resources, 409 for stock/state conflicts, 422 for rule violations and 429 for throttling. Log correlation identifiers; never expose stack traces or secrets. Retry transient external failures with capped exponential backoff and a dead-letter queue.

5.6 Security and privacy considerations

Collect only necessary account, address and order data; publish a retention and deletion policy; isolate admin roles; protect backups and object storage; store payment references rather than card numbers; require explicit consent for optional personalization.

6. Verification and Validation

6.1 Requirement Traceability Matrix

Requirement group	Design evidence	Verification
FR-01–03	Use case; Customer/User classes	TC-01–04
FR-04–06	Catalogue; Product/Variant model	TC-05–08
FR-07–09	Cart/Wishlist/Coupon	TC-09–14
FR-10–12	Activity, sequence, order states	TC-15–18
FR-13–15	Order states; Review	TC-19–22
FR-16–19	Admin, inventory, notification	TC-23–27
NFR-01–12	Deployment/security/quality criteria	TC-28–30 and quality gates

6.2 Test execution convention

ID	Scenario	Preconditions	Test steps	Test data	Expected result	Actual result	Status
TC-01	Registration valid	Guest	Submit unique valid account	new@example.test	Account created	Not Executed	Not Executed
TC-02	Duplicate registration	Existing email	Submit same email	existing@example.test	Conflict; no duplicate	Not Executed	Not Executed
TC-03	Login valid	Active account	Submit credentials	valid credentials	Session issued	Not Executed	Not Executed
TC-04	Invalid credentials	Active account	Submit wrong password	wrong password	Generic denial	Not Executed	Not Executed
TC-05	Search exact	Published items	Search keyword	linen shirt	Matching items	Not Executed	Not Executed
TC-06	Search no result	Catalogue loaded	Search nonsense	zz-no-match	Empty state	Not Executed	Not Executed

All cases below are planned and have not been run. Actual Result and Status are explicitly recorded as Not Executed. Test data is illustrative and shall be provisioned in an isolated environment.

TC-07	Filter combined	Variant data	Apply size/colour/price	M / blue / range	All criteria satisfied	Not Executed	Not Executed
TC-08	Select variant	Active product	Choose size/colour	M / black	Correct SKU/price	Not Executed	Not Executed
TC-09	Wishlist add/remove	Logged in	Toggle item	product UUID	Idempotent list	Not Executed	Not Executed
TC-10	Cart add	In-stock SKU	Add 2 units	SKU qty 2	Correct totals	Not Executed	Not Executed

ID	Scenario	Preconditions	Test steps	Test data	Expected result	Actual result	Status
TC-11	Cart update	Existing cart	Change quantity	2 → 3	Recomputed total	Not Executed	Not Executed
TC-12	Invalid quantity	Existing cart	Submit invalid qty	0 or -1	Rejected	Not Executed	Not Executed
TC-13	Out of stock	Zero available	Add SKU	qty 1	Stock conflict	Not Executed	Not Executed
TC-14	Coupon valid	Eligible cart	Apply code	SAVE10	Correct discount	Not Executed	Not Executed
TC-15	Coupon expired	Eligible cart	Apply expired code	OLD10	No discount	Not Executed	Not Executed
TC-16	Checkout valid	Address/cart	Confirm order	Valid address	Pending order	Not Executed	Not Executed
TC-17	Payment success	Pending order	Send signed success event	Unique event ID	Confirmed once	Not Executed	Not Executed
TC-18	Payment failure	Pending order	Send failure event	Declined payment	Failed; stock released	Not Executed	Not Executed
TC-19	Duplicate webhook	Processed event	Replay same event	Same event ID	No duplicate confirmation	Not Executed	Not Executed
TC-20	Tracking	Shipped order	Read carrier status	Tracking ref	Mapped timeline	Not Executed	Not Executed

ID	Scenario	Preconditions	Test steps	Test data	Expected result	Actual result	Status
TC-21	Cancellation	Eligible order	Request cancel	Before dispatch	Cancelled per policy	Not Executed	Not Executed
TC-22	Return request	Delivered order	Submit within window	Reason code	Return pending	Not Executed	Not Executed
TC-23	Review eligibility	Delivered purchase	Rate product	5 stars	Review accepted	Not Executed	Not Executed
TC-24	Admin login	Admin account	Authenticate	Admin credentials	Admin session	Not Executed	Not Executed
TC-25	Admin product edit	Admin session	Edit product	Valid SKU metadata	Audit and save	Not Executed	Not Executed
TC-26	Inventory adjustment	Admin session	Adjust available stock	+5 units	Atomic audited update	Not Executed	Not Executed
TC-27	Unauthorized admin	Customer session	Call admin endpoint	Customer token	403; no mutation	Not Executed	Not Executed
TC-28	Performance catalogue	Seeded 10k SKUs	Run 200-user load	P95 measurement	P95 ≤ 2 s	Not Executed	Not Executed
TC-29	Payment tampering	Pending order	Send invalid webhook	Bad signature	Reject; unchanged	Not Executed	Not Executed
TC-30	Recovery rehearsal	Backups available	Restore and validate	Latest backup	RPO/RTO target met	Not Executed	Not Executed

6.3 Test strategies

Unit: isolate validation, discount, reservation and state-transition rules using mocked integrations; target NFR-10. Integration: verify database transactions, gateway signatures, carrier status mapping and queue retries with sandbox services. System: run end-to-end customer and admin journeys across supported browsers. User acceptance: stakeholders sign off against FR-01–20 and realistic order scenarios.

Security: RBAC matrix, injection, session fixation, CSRF, credential throttling, webhook replay and sensitive-data exposure checks. Performance: reproducible 200-user workload, warm-up, p95/p99 latency, error rate, resource utilization and bottleneck evidence.

6.4 Boundary value analysis and equivalence partitioning

Field	Valid partition / boundary	Invalid partition / boundary
Cart quantity	1 and 10 inclusive	0, -1, 11, noninteger
Rating	1–5 inclusive	0, 6, null
Coupon expiry	Current instant before expiry	After expiry
Available stock	Requested qty ≤ available	Requested qty > available
Password	At configured minimum and above	Below minimum; breached/weak

7. Feasibility, Risks and Roadmap

7.1 Feasibility study

Technical: conventional web stack and documented external APIs are sufficient; key uncertainty is provider reliability and SKU-level concurrency. Operational: customer and admin journeys fit common retail workflows; training is needed for fulfilment, moderation and refund decisions. Economic: cloud hosting and transaction fees scale with usage; an MVP should prioritize catalogue, cart, checkout and order integrity before optional personalization.

7.2 Risk assessment matrix

Risk	Likelihood	Impact	Mitigation / owner
Overselling under concurrency	Medium	High	Atomic reservation, load tests / backend
Forged/replayed payment callback	Medium	Critical	Signature + event uniqueness / security
Third-party outage	High	Medium	Retry, timeout, graceful status / integration
PII leakage	Medium	Critical	Least privilege, encryption, audits / security
Inaccurate catalogue data	Medium	Medium	Admin validation and review / operations
Delayed delivery updates	High	Low	Show last-sync timestamp / integration

7.3 System limitations

Internet connectivity is required; gateway/carrier availability is outside DFSS control; size recommendations cannot guarantee physical fit; inventory accuracy depends on timely administrator and warehouse updates; tax, shipping and return rules require jurisdiction-specific configuration.

7.4 Future enhancements

Optional future work includes AI-based fashion ranking, virtual fitting room using consented imagery, personalized styling, multilingual content, richer WCAG accessibility validation, size-fit guidance, fraud scoring and warehouse integrations. None is represented as completed implementation.

8.Detailed Operational Contracts and Acceptance Criteria

8.1 Registration and session contract

Registration SHALL normalize email addresses, reject duplicate identities, validate password strength, and create a disabled or verification-pending identity until the selected confirmation workflow succeeds. Session renewal SHALL rotate refresh credentials; logout SHALL revoke the active session. Password reset tokens SHALL be single-use, short-lived, and never logged. Customer and administrator privileges SHALL be evaluated on every protected request rather than inferred from a client-side menu.

8.2 Catalogue and variant integrity

A product represents the merchandising description while a variant represents a purchasable SKU with an independently defined size, colour, price and stock position. The catalogue SHALL exclude unpublished items from guest queries. Search SHALL combine keyword relevance with deterministic filtering and pagination. Variant selection SHALL refresh the displayed price and availability, and SHALL not silently substitute another size or colour.

8.3 Transaction boundaries and concurrency

At checkout the server SHALL re-read cart contents, coupon eligibility, delivery coverage and current SKU pricing. A single atomic reservation SHALL prevent two concurrent purchases from allocating the same last unit. Each reservation SHALL have an expiry timestamp and a unique order association. A gateway callback SHALL be authenticated, checked for matching amount and currency, and deduplicated by provider event identifier. Replayed success events SHALL never create a second order or reduce inventory twice.

8.4 Payment exception matrix

Condition	System action	Customer-visible result
Gateway timeout	Retain pending state until reconciliation; do not assume failure	Payment is being verified
Explicit decline	Mark attempt failed; release reservation per policy	Retry using supported method
Duplicate success webhook	Return acknowledged response without additional mutation	Single confirmed order
Amount or currency mismatch	Quarantine event; raise operational alert	Order remains pending review
Refund request accepted	Create traceable refund attempt and reconcile provider reference	Refund processing status

8.5 Order fulfilment and return rules

An order SHALL expose a chronological event history with actor, event type, timestamp and correlation identifier. Only authorized fulfilment users may move an order from processing to packed, shipped or delivered, subject to carrier evidence. Cancellation SHALL be blocked after dispatch unless a configured exception applies. A return request SHALL validate ownership, delivery state, policy window and permitted product category; approval SHALL not itself imply that a refund has settled.

8.6 Notification delivery and auditability

Notifications SHALL be generated from committed domain events rather than speculative UI actions. Message retries SHALL use stable idempotency keys, and a delivery failure SHALL not roll back an already paid order. Administrators SHALL be able to inspect a masked audit trail of inventory adjustments, product edits, order status changes and refund decisions. Logs SHALL exclude passwords, payment credentials and full personal addresses.

8.7 Accessibility and responsive interface acceptance

Primary flows SHALL support keyboard navigation, visible focus, semantic labels, descriptive validation errors and screen-reader announcement of cart and payment status. Layouts SHALL be evaluated at narrow mobile widths without horizontal overflow of the checkout form. Colour SHALL not be the sole indicator of availability or error. Product images SHALL provide useful alternative descriptions where they communicate purchase-relevant information.

8.8 Data retention, backup and recovery acceptance

The system maintains documented retention schedules for account, transaction, audit, and diagnostic records according to applicable policies. Daily encrypted backups are stored securely and tested in an isolated environment to verify data integrity, order totals, and payment reconciliation. Recovery procedures define responsible operators, escalation limits, and customer communication guidelines during service disruptions.

8.9 Acceptance scenarios and release gates

Gate	Evidence required	Pass condition
Functional baseline	Executed TC-01 through TC-27 and requirement review	No unresolved critical checkout or authorization defects
Payment integrity	Signed callback, replay, mismatch and timeout scenarios	No duplicate fulfilment or unverified confirmation
Performance	Repeatable 200-user workload and latency report	Published p95 thresholds satisfied
Security	RBAC, injection, secrets and webhook assessment	No open critical or high-risk finding
Recovery	Backup restore and reconciliation record	Configured RPO/RTO met
Accessibility	Keyboard and assistive technology walkthrough	No blocking issue in core purchase flow

8.10 Proposed release sequencing

Iteration 1: identities, roles, catalogue and variants. Iteration 2: search, wishlist, cart and inventory constraints. Iteration 3: order state machine, coupons, payment adapter and notification outbox. Iteration 4: carrier tracking, cancellations, returns, reviews and reporting. Iteration 5: end-to-end security, performance, accessibility and recovery acceptance. Virtual try-on and AI personalization remain separately estimated enhancements and are not release-blocking baseline functions.

8.11 API resource and authorization specification

Resource / operation	Authorized principal	Required behavior
POST /accounts	Guest	Validate unique email; return safe account identifier
POST /sessions	Registered user	Rate-limit attempts; issue bounded session
GET /products	Public	Paginate published products and active variants
PUT /cart/items/{id}	Customer owner	Validate variant and integer quantity
POST /orders	Customer owner	Reprice, reserve stock and create pending order
POST /payments/webhook	Verified provider	Authenticate signature and deduplicate event
GET /orders/{id}	Owner or privileged admin	Mask sensitive fields and return event timeline
PATCH /admin/inventory/{sku}	Inventory administrator	Audit actor, reason and stock delta
POST /returns	Order owner	Check return policy and persist request

API requests and responses use versioned schemas with consistent field names, explicit currency codes, and standardized error identifiers. Collection endpoints support limited page sizes and predictable sorting. Data-modifying operations use idempotency keys to prevent duplicate transactions. Unknown fields are validated consistently to prevent unauthorized access or privilege escalation..

8.12 Price, coupon and total calculation rules

The Digital Fashion Shopping System calculates the final checkout amount on the server side using fixed order-line records to ensure accuracy and prevent unauthorized price modifications.
The subtotal is calculated by multiplying the quantity of each product by its unit price and adding the resulting amounts using the smallest currency unit. Eligible product-level discounts are applied first, followed by order-level coupon discounts according to the configured coupon rules.

authorized payment amount SHALL equal the persisted order total, and a discrepancy SHALL prevent confirmation.

Calculation control	Acceptance rule	Failure behavior
Unit price	Taken from active SKU at order creation	Reprice cart and request confirmation
Coupon window	Current server time within valid interval	Reject coupon without changing cart
Coupon usage	Enforce per-user and global limits atomically	Return eligibility error
Shipping charge	Use selected supported destination/service	Show unavailable delivery option
Tax/rounding	Use configured policy and minor units	Block checkout if policy unavailable
Total integrity	Persist breakdown and final amount	Reject mismatched gateway callback

8.13 Inventory ledger and reconciliation

The inventory management system maintains accurate stock records by distinguishing between physical stock on hand, active reservations, allocated stock, and available-to-sell quantities. The available-to-sell quantity must always remain nonnegative to prevent overselling and maintain inventory accuracy.The system handles reservation expiration and stock release operations safely, even when scheduled jobs execute multiple times. Every manual inventory adjustment is recorded with essential details, including the reason for modification, responsible user or administrator, previous stock quantity, updated quantity, and timestamp.A scheduled reconciliation process identifies inventory inconsistencies, including orphaned reservations, paid orders without allocated stock, and completed deliveries with incorrect fulfilment statuses. When discrepancies are detected, the system generates alerts for administrators instead of automatically overwriting existing records.

8.14 Detailed data ownership and deletion policy

The system provides customers with the ability to view and update their personal profiles and saved delivery addresses while maintaining necessary order records for legal and accounting purposes.
Historical order information is preserved according to the defined data retention policy, with unnecessary exposure of personal information minimized. Product images are stored in private or controlled-access storage, with file size and content-type validation to ensure security and prevent unauthorized uploads.
Customer reviews maintain their moderation status and author references to support accountability and content management. Account deletion requests are processed according to the configured data retention policy. Where historical order records must be retained, an appropriate anonymization strategy protects customer privacy while preserving valid database relationships.

.

8.15 Service-level measurement method

The performance and availability of the Digital Fashion Shopping System are evaluated using defined testing procedures and measurable service-level indicators.Application response time is measured at the public HTTPS interface using representative request payloads, a predefined product catalogue, and a consistent workload profile.Performance test reports include information about the hardware configuration, network conditions, concurrent users, system warm-up period, test duration, response time percentiles (p50, p95, and p99), throughput, and error percentage.System availability is calculated through automated monitoring checks performed at fixed intervals, excluding only documented planned maintenance periods.Monitoring and alert thresholds cover important operational conditions, including increased API error rates, notification queue delays, pending payment reconciliation tasks, database resource saturation, and failed backup operations. These measurements help identify performance issues and support reliable system operation.

8.16 Administrator workflows and separation of duties

The administration console provides secure access to authorized administrators through authenticated accounts. Important administrative activities and system modifications are recorded to maintain accountability and support auditing.Before publishing a product, administrators are required to provide and validate essential information, including product category, title, SKU variants, pricing details, and product images.
Inventory adjustments require a valid reason, ensuring that stock modifications can be tracked and reviewed when necessary.Where sufficient administrative staff are available, refund initiation and refund approval are handled by separate authorized individuals to improve financial control and reduce the risk of unauthorized transactions.
The analytics dashboard provides administrators with summarized information about sales performance, order counts, refunds, and low-stock products without exposing confidential customer information.
Data export functions are restricted according to administrator permissions, and all export activities are logged for security

monitoring and future reference.

8.17 Error catalogue and customer recovery

Error code	Trigger	Required response
AUTH_INVALID	Invalid login secret	Generic error; no account enumeration
SKU_UNAVAILABLE	Stock below requested amount	Show current availability and allow edit
Error code	Trigger	Required response
COUPON_INELIGIBLE	Rule or expiry failure	Explain non-sensitive eligibility condition
PAYMENT_PENDING	No final verified callback	Offer status refresh; do not duplicate order
PAYMENT_DECLINED	Verified provider decline	Permit safe retry with new attempt
ORDER_STATE_CONFLICT	Invalid transition	Keep original state and log correlation ID
RATE_LIMITED	Request threshold exceeded	Return retry guidance without leaking limits
INTERNAL_ERROR	Unexpected application fault	Safe message; server-side trace only

8.18 Threat Model and Trust Boundaries

The Digital Fashion Shopping System (DFSS) incorporates security controls to protect customer information, payment transactions, and administrative operations. Since the browser is considered an untrusted environment, the application must validate all submitted prices, quantities, customer identities, and order references on the server side.

Payment gateways and shipping providers operate as external services with separate trust boundaries. Their callbacks must be authenticated and associated only with existing transaction or order records.

Administrative operations are protected through server-side Role-Based Access Control (RBAC), with multi-factor authentication recommended for additional security. Database credentials are restricted to the permissions required for each operation, while production secrets are stored securely outside the source code.

Potential security threats include credential stuffing, repeated coupon guessing, order-ID enumeration, malicious file uploads, forged payment callbacks, and repeated inventory reservation attempts. Appropriate validation, authentication, authorization, rate limiting, and monitoring mechanisms are required to reduce these risks.

8.19 Observability and Operational Response

The system includes monitoring and operational response mechanisms to maintain reliability and identify transaction failures.

Each checkout attempt is assigned a unique correlation identifier that connects the corresponding order, payment, and notification records. This enables administrators and developers to trace transactions across different components.

The monitoring system tracks important operational metrics, including:

- Checkout completion rate
- Distribution of payment failures
- Inventory reservation conflicts
- Notification queue delays
- Delivery provider synchronization delays
- API error ratios

Structured application logs are retained according to the defined retention policy, with sensitive personal information removed or masked.

Payment reconciliation alerts contain relevant order references and event identifiers without exposing confidential payment credentials.

Operational runbooks document incident escalation, rollback procedures, external provider outages, system recovery, and post-recovery verification.

8.20 Data Migration and Configuration Controls

The initial product catalogue import process validates the accuracy and consistency of product information before storing records in the database.

Validation includes category references, unique Stock Keeping Unit (SKU) values, supported sizes, currency information, product image references, and nonnegative inventory quantities.

Invalid records are separated from valid data and stored with row-level error details for correction. Partial imports must not produce duplicate or ambiguous product variants.

The application also maintains environment-specific configuration settings, including payment and shipping provider endpoints, return windows, supported delivery regions, tax policies, coupon rules, inventory reservation timeouts, and notification templates.

Configuration changes are reviewed, version-controlled, and tested before being introduced into the production environment.

8.21 Expanded Verification Matrix

The expanded verification matrix identifies the testing techniques and evidence required to validate important system functions.

Area	Testing Technique	Evidence Artifact
Account Validation	Equivalence classes and boundary value analysis	Field validation test report
Cart Concurrency	Parallel checkout for the final available unit	Reservation transaction trace
Gateway Integration	Signed, unsigned, and replayed callback testing	Webhook verification log
Order States	Model-based state transition testing	State transition matrix
Permissions	Role-resource negative testing	Authorization results
Accessibility	Keyboard and screen reader testing	Accessibility issue register
Recovery	Restoration of encrypted database backup	Restore checklist and timing
Carrier Integration	Delayed and out-of-order tracking events	Event mapping report

Each test result is linked to its corresponding requirement identifier, execution date, testing environment, and defect record.

Failed acceptance criteria remain unresolved until the necessary corrections are implemented and successful retesting is documented.

The status Not Executed indicates that the test cases have been defined but have not yet been performed. It does not imply that the proposed system has been deployed or validated.

9.22 Detailed Test Data Design

A structured test dataset is prepared to verify system behavior under normal, invalid, and exceptional conditions.

Test identities include an unregistered guest, a verified customer, a locked customer account, an administrator, and a customer with an existing delivered purchase.

Product test records include active products with multiple size and colour variants, unpublished products, out-of-stock items, low-stock items, and discontinued products.

Coupon test records cover valid, expired, exhausted, minimum-spend-ineligible, and customer-restricted coupons.

Payment testing includes approved, declined, timed-out, duplicate, and forged callback scenarios. Synthetic customer names and addresses are used to protect personal information.

Dataset	Representative Values	Coverage Purpose
Quantity	-1, 0, 1, 10, 11, decimal, null	Boundary and type validation

Stock	0, 1, 2, concurrent requests for one unit	Availability and race-condition prevention
Order Status	Pending, confirmed, shipped, delivered	Valid and invalid state transitions
Coupon	Active, expired, exhausted, restricted	Coupon rule validation
Identity	Guest, owner, another customer, administrator	Horizontal and vertical authorization
Provider Event	Valid, replayed, invalid signature, mismatch	Authenticity and idempotency

These datasets help evaluate input validation, transaction integrity, access control, and error handling.

9.23 Interface-Level Contract Examples

The system defines consistent interface contracts to support communication between the frontend, backend, and external services.

Catalogue Response: The catalogue API provides product identifiers, display titles, categories, media URLs, available size and colour variants, prices, and currency information. Confidential supplier costs and unpublished internal notes are excluded from public responses.

Cart Response: The shopping cart API returns the cart identifier, line identifiers, selected SKU, quantity, unit price, estimated total, and availability warnings.

Order Response: The order API provides the order reference, purchased product snapshots, charge breakdown, payment status, fulfilment status, and customer-safe tracking information.

Payment Webhook: Incoming payment callbacks are processed only after the authenticity of the original signed payload has been verified.

These contracts support consistent data exchange and reduce the possibility of unauthorized information disclosure.

9.24 Change Control and Requirement Governance

The project follows a structured change management process to maintain consistency between requirements, design documents, implementation, and testing.

Every change to the approved requirements baseline includes a unique change identifier, affected requirement IDs, justification, priority, stakeholder approval, and an assessment of its impact on UML diagrams, database schemas, test cases, and release planning.

Optional features are managed separately from committed baseline requirements to avoid unapproved scope changes.

The Requirements Traceability Matrix (RTM) is updated whenever a requirement is introduced, modified, or removed.

Any contradiction between written requirements and UML diagrams is resolved through a documented review decision before implementation.

Requirements are written using clear, unambiguous language and include measurable or verifiable acceptance conditions.

9.25 Project Deliverables and Exit Checklist

The project deliverables provide the documentation and verification evidence required for academic evaluation and future implementation.

Deliverable	Review Criterion
Requirements Baseline	Numbered functional and measurable quality requirements
UML Model Set	Use case, activity, class, sequence, state, deployment, and component diagrams
Data Model	ER diagram, database constraints, ownership rules, and data dictionary
Implementation Design	API contracts, authorization rules, and transaction boundaries

Test Plan	Traceable test cases, datasets, and negative test scenarios
Operational Plan	Monitoring, backup, reconciliation, and incident response procedures
Release Approval	Stakeholder acceptance and no unresolved critical defects

The academic review verifies that all diagrams use terminology consistent with the requirements.

Special attention is given to payment confirmation, inventory reservation, order status transitions, and return handling to ensure consistency across the UML models and system design.

This document describes a proposed software platform. It does not claim that third-party integrations, acceptance tests, or optional artificial intelligence features have already been implemented.

9.26 Acceptance Criteria for Exceptional Shopping Journeys

This section describes how the proposed system is expected to respond to exceptional shopping scenarios.

9.26.1 Product Size or Colour Modification

When a customer changes the selected size or colour of a product already present in the shopping cart, the system resolves the corresponding SKU and refreshes its price and stock availability.

If the new variant has a different price, the customer is asked to confirm the change before proceeding.

The modification affects only the selected cart item and does not alter unrelated cart lines. If the requested variant is unavailable, the original selection is preserved.

9.26.2 Concurrent Inventory Reservation

When two customers attempt to purchase the final available unit of a product simultaneously, only one inventory reservation can be committed successfully.

The unsuccessful customer receives an availability conflict message, and the shopping cart is refreshed to reflect the current stock level.

Database transactions and integrity constraints prevent overselling rather than relying solely on frontend stock indicators.

9.26.3 Payment Completion After Browser Closure

If a customer closes the browser after completing payment, the application continues processing the payment through the authenticated gateway callback.

The verified payment event is reconciled with the existing pending order.

When the customer returns to the application, the correct order status is displayed without initiating a duplicate checkout.

Order history remains accessible independently of email or push notification delivery.

9.26.4 Out-of-Order Delivery Tracking Updates

When a delivery provider sends tracking updates in an incorrect sequence, the system preserves the original carrier event and applies the defined order-status mapping rules.

Inconsistent updates are flagged for administrative review when necessary.

For example, a delayed *Packed* event cannot replace an order status that has already reached *Shipped*.

The customer-facing tracking timeline displays the latest trusted delivery status along with its synchronization time.

9.26.5 Return Approval and Refund Processing

When a customer return is approved, the refund process verifies that the original payment was successfully captured.

The system prevents the cumulative refunded amount from exceeding the original captured amount.

Each refund attempt is recorded with its corresponding payment provider reference.

If a refund attempt fails, the transaction remains available for reconciliation and is not incorrectly marked as fully refunded.

9.26.6 Modification of Products in Historical Orders

When an administrator edits or unpublishes a product, the changes do not affect previously completed orders.

Historical order records preserve the original product description, selected variant, unit price, and charged amount.

Unpublished products are removed from public catalogue discovery, while authorized customers retain access to their previous purchase details.

9.26.7 Notification Provider Unavailability

If the external notification provider becomes unavailable, the application stores the committed notification event in a queue for later delivery.

Failed notifications are retried using a bounded backoff strategy.

When the maximum retry limit is reached, an operational alert is generated for investigation.

Notification failures do not reverse verified payments, modify confirmed orders, or trigger additional inventory deductions.

9.26.8 Session Expiration During Checkout

If a customer session expires during checkout, the application requests reauthentication while preserving the safe shopping cart context.

Sensitive payment credentials are not retained, and unauthenticated requests cannot modify orders or access another customer's address information.

After successful reauthentication, the server revalidates product availability, prices, quantities, discounts, and other checkout conditions before allowing the transaction to continue.

10. Conclusion and References

10.1 Conclusion

The Digital Fashion Shopping System (DFSS) is a proposed online fashion shopping platform designed to provide a secure, reliable, and convenient purchasing experience.

The system covers essential e-commerce operations, including customer registration and authentication, product catalogue management, size and colour selection, shopping cart management, secure checkout, payment integration, inventory control, order tracking, returns, refunds, and administrative management.

The Software Requirements Specification establishes the functional requirements, quality expectations, security controls, data contracts, UML models, and verification approach required for systematic software development.

Special consideration has been given to transaction integrity, inventory concurrency, payment authentication, customer privacy, and reliable communication with external service providers.

The requirements and design artifacts provide an academic baseline for future implementation, testing, and deployment. The project documentation does not represent a claim that the complete platform or its external integrations have already been developed and validated.

10.2 References

[1] IEEE Std 830-1998, *Recommended Practice for Software Requirements Specifications*.

[2] ISO/IEC/IEEE 29148:2018, *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*.

[3] Object Management Group (OMG), *Unified Modeling Language Specification*, Version 2.5.1.

[4] OWASP, *Application Security Verification Standard (ASVS)*.

[5] World Wide Web Consortium (W3C), *Web Content Accessibility Guidelines (WCAG) 2.2*.

[6] PCI Security Standards Council, *Payment Card Industry Data Security Standard (PCI DSS)*.